Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ

КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

СЕМИНАР №3

ДИСЦИПЛИНА: «Программирование на основе классов и шаблонов»

Выполнил: студент гр. ИУ5-24Б _____ (Лимаренко Д. А.)
(Подпись) (Ф.И.О.)

Проверил: _____ (Волосова А. В.)
(Подпись) (Ф.И.О.)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:
- Оценка:

2026 г.

Задание: спроектировать и реализовать на C++ структуру классов, описывающую заданную предметную область. В коде должны быть продемонстрированы:

1. Иерархия наследования (минимум 3 класса в иерархии, если позволяет область).
2. Ассоциация (один класс хранит ссылку или указатель на другой).
3. Агрегация (класс содержит коллекцию других объектов, которые могут существовать отдельно).
4. Композиция (класс содержит объекты, время жизни которых жестко привязано к нему).
5. Делегирование (один метод класса передает выполнение части работы методу другого класса).

Предметная область: вариант №17 − школа.

Выполнение работы

Был создан репозиторий на GitHub: https://github.com/DariaLimarenko/SchoolProject.

В рамках данной работы была дана предметная область «Школа». В школе основными сущностями являются ученики, учителя, учебные материалы, оценки и сама организация, объединяющая все эти элементы.

Целью моделирования данной предметной области является создание программной структуры, которая отражает реальные связи между объектами и их отношения. В данной работе акцент сделан на представлении ключевых участников образовательного процесса — учеников и учителей, а также на системе хранения и обработки оценок.

В школьной системе ученик получает знания, выполняет задания и получает оценки, которые фиксируются в дневнике. Учитель, в свою очередь, ведёт занятия по определённому предмету и может выступать классным руководителем для группы учеников.

Таким образом, предметная область включает в себя следующие основные сущности: человек, ученик, учитель, дневник и школа. Эти сущности имеют свои свойства, поведение и связи друг с другом.

В процессе выполнения работы была разработана иерархия классов, отражающая структуру предметной области. Каждый класс отвечает за определённую часть логики и имеет чётко определённые обязанности.

Класс Person является базовым классом в данной системе. Он описывает общие свойства, характерные для любого человека, а именно имя и возраст. Также в этом классе реализован метод вывода информации об объекте. Использование базового класса позволяет избежать дублирования кода и объединить общие характеристики для учеников и учителей.

Класс Teacher наследуется от класса Person и представляет учителя. В дополнение к унаследованным свойствам он содержит информацию о предмете, который преподаёт учитель. В классе переопределён метод вывода информации, чтобы учитывать специфику учителя. Таким образом, данный класс расширяет функциональность базового класса и добавляет профессиональные характеристики.

Класс Student также наследуется от класса Person и представляет ученика. Он содержит уникальный идентификатор ученика, а также объект дневника, в котором хранятся оценки. Кроме того, класс хранит указатель на учителя, который является классным руководителем. В классе реализованы методы для добавления оценок, вычисления среднего балла, вывода информации и отображения содержимого дневника. Этот класс является центральным элементом модели, так как взаимодействует с несколькими другими классами.

Класс Diary отвечает за хранение и обработку оценок ученика. Он содержит коллекцию оценок, реализованную с помощью контейнера vector. В данной работе используется школьная система оценивания от 2 до 5. Класс предоставляет методы для добавления оценок, вычисления среднего значения и вывода списка оценок. Выделение этого класса позволяет отделить логику работы с оценками от логики работы ученика.

Класс School представляет собой организацию, объединяющую учеников и учителей. Он содержит название школы, а также списки учеников и учителей. В классе реализованы методы для добавления объектов в эти списки и для вывода информации обо всех участниках. Также предусмотрен метод для отбора учеников

с высоким средним баллом. Этот класс выполняет роль управляющего элемента системы.

В разработанной программе реализованы основные виды отношений между классами, характерные для объектно-ориентированного программирования.

Иерархия наследования представлена классом Person и его производными классами Student и Teacher. Это отношение типа «is-a», которое означает, что ученик и учитель являются частными случаями человека. Наследование позволяет использовать общие свойства и методы базового класса, а также расширять их в производных классах.

Ассоциация реализована между классами Student и Teacher. Каждый ученик связан с учителем через указатель на классного руководителя. При этом оба объекта могут существовать независимо друг от друга, что делает связь слабой. Такая связь отражает реальную ситуацию, когда ученик может сменить классного руководителя, а учитель может работать с разными учениками.

Агрегация реализована в классе School. Школа содержит коллекции учеников и учителей, но не создаёт их самостоятельно. Объекты создаются отдельно и передаются в школу. Это означает, что ученики и учителя могут существовать независимо от школы, а школа лишь объединяет их в рамках общей структуры.

Композиция реализована между классами Student и Diary. Каждый ученик содержит внутри себя объект дневника, который не может существовать отдельно. Время жизни дневника полностью зависит от времени жизни ученика. Это отражает сильную связь между объектами, при которой один объект является частью другого.

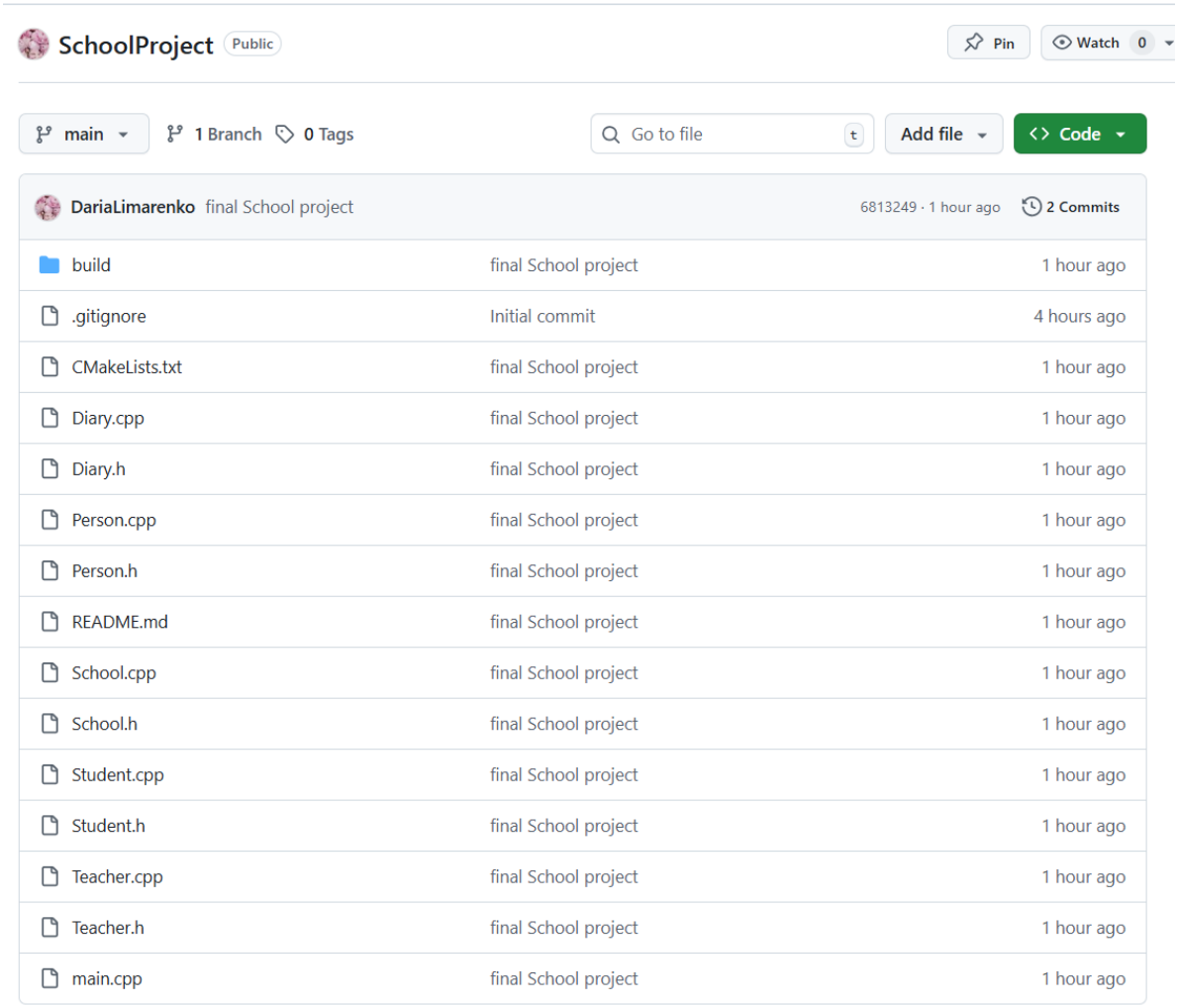
Делегирование реализовано в классе Student. При добавлении оценок или вычислении среднего балла ученик не выполняет эти операции самостоятельно, а передаёт их объекту Diary. Это позволяет разделить обязанности между классами и сделать структуру программы более логичной и удобной для сопровождения.

Вывод работы программы в терминале VS Code представлен на рисунке 1.

```
--- Список учителей школы Средняя школа №1 ---
Учитель: Иванова Мария Петровна, возраст: 40, предмет: Математика
Учитель: Сидоров Алексей Викторович, возраст: 35, предмет: Информатика

--- Список учеников школы Средняя школа №1 ---
Ученик: Петров Иван, возраст: 15, ID: 101, классный руководитель: Иванова Мария Петровна
Ученик: Смирнова Анна, возраст: 16, ID: 102, классный руководитель: Иванова Мария Петровна
Ученик: Козлов Максим, возраст: 15, ID: 103, классный руководитель: Сидоров Алексей Викторович

--- Дневники учеников ---
Дневник ученика Петров Иван: Оценки: 5 4 5 , средний балл = 4.67
Дневник ученика Смирнова Анна: Оценки: 3 2 4 , средний балл = 3.00
Дневник ученика Козлов Максим: Оценки: 5 5 4 , средний балл = 4.67

--- Ученики со средним баллом >= 4 ---
Ученик: Петров Иван, возраст: 15, ID: 101, классный руководитель: Иванова Мария Петровна
Ученик: Козлов Максим, возраст: 15, ID: 103, классный руководитель: Сидоров Алексей Викторович
```

Рисунок 1 – Вывод работы программы в терминале VS Code

Оформление репозитория на GitHub представлен на рисунках 2-6.



README

SchoolProject

Описание проекта

Данный проект представляет собой объектно-ориентированную модель предметной области «Школа», реализованную на языке C++. В программе показано применение основных принципов объектно-ориентированного проектирования: наследования, ассоциации, агрегации, композиции и делегирования. Проект разработан в многофайловом формате, где каждый класс вынесен в отдельные файлы заголовка и реализации.

Программа моделирует школьную систему, в которой есть учителя, ученики, дневники и школа как объединяющий объект. В процессе работы создаются объекты учителей и учеников, ученикам назначаются классные руководители, в дневники добавляются оценки, после чего программа выводит информацию обо всех участниках и показывает учеников с высоким средним баллом.

Структура проекта

Проект состоит из следующих файлов:

- `Person.h`, `Person.cpp`
- `Teacher.h`, `Teacher.cpp`
- `Student.h`, `Student.cpp`
- `Diary.h`, `Diary.cpp`
- `School.h`, `School.cpp`
- `main.cpp`
- `CMakeLists.txt`

Файл `main.cpp` содержит точку входа в программу. В нём создаются объекты, задаются связи между ними и выполняется демонстрация работы всей модели.

Описание классов

Класс `Person`

Класс `Person` является базовым классом в данной иерархии. Он описывает общие свойства человека в школьной системе. В этом классе хранятся поля `name` и `age`, то есть имя и возраст. Также в классе определён метод `showInfo()`, который предназначен для вывода основной информации об объекте, и метод `getName()`, который возвращает имя.

Этот класс нужен для того, чтобы не дублировать одинаковые поля и методы в классах ученика и учителя. Поскольку и ученик, и учитель являются людьми, именно `Person` выступает для них общей основой.

Класс `Teacher`

Класс `Teacher` наследуется от класса `Person` и представляет учителя. Помимо полей, полученных от базового класса, он содержит дополнительное поле `subject`, в котором хранится предмет, преподаваемый учителем.

Метод `showInfo()` в этом классе переопределён, чтобы выводить не только имя и возраст, но и предмет учителя. Таким образом, класс `Teacher` расширяет общий класс `Person` и добавляет ему профессиональную специфику.

Класс `Student`

Класс `Student` также наследуется от класса `Person` и представляет ученика. Он содержит поле `studentId`, в котором хранится идентификатор ученика, объект `Diary`, который играет роль дневника, и указатель на объект `Teacher`, обозначающий классного руководителя.

У класса есть метод `setClassTeacher()`, с помощью которого ученику назначается классный руководитель. Также в классе реализованы методы `addGrade()` и `getAverageGrade()`. Эти методы не выполняют всю работу сами, а передают её объекту дневника. Кроме того, у класса есть метод `showDiary()`, который позволяет вывести оценки ученика и его средний балл, а также переопределённый метод `showInfo()`.

Класс `Student` является центральным элементом модели, так как он участвует сразу в нескольких отношениях: наследовании, ассоциации, композиции и делегировании.

Класс `Diary`

Класс `Diary` описывает дневник ученика. Внутри него хранится список оценок в виде контейнера `vector<int>`. В проекте используется школьная система оценивания от 2 до 5.

Метод `addGrade()` добавляет новую оценку в дневник, предварительно проверяя, что она находится в допустимом диапазоне. Метод `calculateAverage()` вычисляет средний балл ученика по всем оценкам. Метод `showGrades()` выводит список всех оценок.

Этот класс отвечает только за работу с оценками. Благодаря этому логика хранения и обработки оценок не смешивается с логикой самого ученика, что делает программу более понятной и правильно организованной.

Класс `School`

Класс `School` описывает школу как объект, объединяющий учеников и учителей. В нём хранятся название школы, список указателей на учеников и список указателей на учителей.

Методы `addStudent()` и `addTeacher()` позволяют добавлять в школу уже существующие объекты. Методы `showAllStudents()` и `showAllTeachers()` выводят списки всех учеников и учителей. Метод `showExcellentStudents()` показывает учеников, чей средний балл не ниже 4.0.

Класс `School` нужен для организации общей структуры программы. Он объединяет участников образовательного процесса и предоставляет методы для работы со всей школьной системой в целом.

Отношения между классами

Иерархия наследования

В проекте реализована иерархия наследования, построенная на основе базового класса `Person`. От него наследуются классы `Student` и `Teacher`. Это означает, что ученик и учитель являются частными случаями человека и имеют общие свойства, такие как имя и возраст.

Наследование позволяет вынести общие данные и поведение в один базовый класс, а в производных классах оставить только специфические особенности. Благодаря этому код становится компактнее, понятнее и удобнее для расширения.

Ассоциация

Ассоциация реализована между классами `Student` и `Teacher`. Каждый ученик может быть связан с учителем через указатель на классного руководителя. Эта связь не является жёсткой: учитель и ученик могут существовать отдельно друг от друга.

Такое отношение показывает обычную связь между объектами, когда один объект знает о другом, но не управляет его временем жизни. В данной программе ученик просто хранит ссылку на учителя, назначенного ему как классный руководитель.

Агрегация

Агрегация реализована в классе `School`. Школа содержит списки учеников и учителей, но не создаёт их внутри себя. Объекты создаются отдельно в `main.cpp`, а затем передаются в школу через методы добавления.

Это означает, что ученики и учителя могут существовать независимо от школы. Школа только объединяет их в рамках общей модели. Именно поэтому такое отношение считается агрегацией, а не композицией.

Композиция

Композиция реализована между классами `Student` и `Diary`. Каждый объект ученика содержит внутри себя объект дневника. Дневник не создаётся отдельно и не существует сам по себе вне ученика.

Такое отношение является сильным, потому что время жизни дневника полностью зависит от времени жизни ученика. Когда ученик создаётся, создаётся и его дневник. Когда ученик уничтожается, вместе с ним уничтожается и дневник.

Делегирование

Делегирование в проекте реализовано в классе `Student`. Когда ученику нужно добавить оценку или вычислить средний балл, он не делает это самостоятельно, а передаёт выполнение этих действий объекту `Diary`.

Например, метод `Student::addGrade()` вызывает метод `Diary::addGrade()`, а метод `Student::getAverageGrade()` вызывает `Diary::calculateAverage()`. Это позволяет разделить обязанности между классами и избежать избыточного кода.

Логика работы программы

После запуска программа создаёт несколько объектов учителей и учеников. Затем каждому ученику назначается классный руководитель. После этого в дневники учеников добавляются оценки по школьной шкале от 2 до 5.

Далее создаётся объект школы, в который добавляются все учителя и ученики. После этого программа выводит список учителей, список учеников, дневники с оценками и средним баллом, а затем показывает учеников, у которых средний балл не ниже 4.0.

Таким образом, программа не только демонстрирует структуру классов, но и наглядно показывает, как между ними работают разные виды отношений.

Сборка и запуск

Проект собирается с помощью **CMake**.

Команда для сборки и запуска:

```
rm -rf build && cmake -B build && cmake --build build && ./build/school_app
```

Рисунки 2-6 – Оформление репозитория на GitHub

Вывод: в ходе выполнения работы была разработана объектно-ориентированная модель предметной области «Школа». В процессе разработки были применены основные принципы объектно-ориентированного программирования, включая наследование, ассоциацию, агрегацию, композицию и делегирование.

Каждый класс в системе выполняет чётко определённую роль, а взаимодействие между классами построено с учётом реальных связей в предметной области. Это позволило создать логичную и расширяемую структуру программы.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Быков, А. Ю. Решение задач на языках программирования Си и Си++ : методические указания / А. Ю. Быков. — Москва : МГТУ им. Н.Э. Баумана, 2017. — 248 с. — ISBN 978-5-7038-4577-6. — Текст : электронный // Лань : электроннобиблиотечная система. — URL: https://e.lanbook.com/book/103505

2. Каширин, И. Ю. От Си к Си++ : учебное пособие / И. Ю. Каширин, В. С. Новичков. — 2-е изд., стер. — Москва : Горячая линия-Телеком, 2012. — 334 с. — ISBN 978-5-9912-0259-6. — Текст : электронный // Лань : электроннобиблиотечная система. — URL: https://e.lanbook.com/book/5161

3. Быков А. Ю. Решение задач на языках программирования Си и Си++ : метод. указания к выполнению лаб. работ / Быков А. Ю. ; МГТУ им. Н. Э. Баумана. - М. : Изд-во МГТУ им. Н. Э. Баумана, 2017. - 244 с. : ил. - ISBN 978-570384577-6.

4. Иванова Г. С., Ничушкина Т. Н. Объектно-ориентированное программирование : учебник для вузов / Иванова Г. С., Ничушкина Т. Н. ; общ. ред. Иванова Г. С. - М. : Изд-во МГТУ Н.Э.Баумана, 2002. http://progbook.ru/technologiya-programmirovaniya/582-ivanova-tehnologiyaprogrammirovaniya.html

5. Иванова Г. С., Ничушкина Т. Н. Объектно-ориентированное программирование : учебник для вузов / Иванова Г. С., Ничушкина Т. Н. ; общ. ред. Иванова Г. С. - М. : Изд-во МГТУ им. Н. Э. Баумана, 2014. - 455 с. : ил. - Библиогр.: с. 450. - ISBN 978-5-7038-3921-8.

6. Подбельский В.В. Язык Си++: Учебное пособие. – М.: Финансы и статистика, 2003. http://progbook.ru/c/737-podbelskii-programmiovanie-na-yazyke-si.html.

7. Акулов О.А., Медведев Н.В. Информатика. Базовый курс. – М.: Омега-Л, 2006. http://razym.ru/naukaobraz/obrazov/151874-akulov-oa-medvedevnvinformatika-bazovyy-kurs.html

8. Вычислительные методы и программирование. МГУ им. М.В. Ломоносова. ISSN 1726-3522.Журнал входит в 1-й уровень Белого списка научных журналов Минобрнауки России. https://num-meth.ru/index.php/journal/index

Дополнительные материалы

1. Иванова Г.С. Технология программирования: Учебник для вузов. – М.: Изд. МГТУ им. Ахо А.В., Хопкрофт Д.Э., Ульман Д.Д. Структуры данных и алгоритмы. – М., Вильямс, 2003. http://razym.ru/naukaobraz/obrazov/181547ahoa-ulman-d-hopkroft-d-struktury-dannyh-i-algoritmy.html
2. Дейтел Х.М., Дейтел П.Дж. Как программировать на С++. – М.: Бином, 2001.
3. Т. Кормен Т., Лейзерсон Ч., Ривест Р. Алгоритмы: построение и анализ. – М. МЦНМО, 2005. http://rutracker.org/forum/viewtopic.php?t=533181
4. Джосьютис Н. С++ Стандартная библиотека для профессионалов. – СПб.: Питер, 2004. http://progbook.ru/c/178-dzhosyutis_c_standartnaya_biblioteka.html
5. Подбельский В.В. Стандартный Си++: Учебное пособие. – М.: Финансы и статистика, 2008.
6. Объектно-ориентированное программирование в С++: пер. с англ. / Лафоре Р. - 4-е изд. - СПб.: Питер, 2004. - 923 с. - (Классика computer science). - ISBN 594723302-9.
7. Т. Кормен Т., Лейзерсон Ч., Ривест Р. Алгоритмы: построение и анализ. – М. МЦНМО, 2005.
8. Г. Шилдт. С++. Базовый курс, 3-е издание: Пер с англ. – М.: Издательский дом «Вильямс», 2011. – 624 с.
9. Павловская Т. А. С/С++. Программирование на языке высокого уровня: учебник для вузов / Павловская Т. А. - СПб.: Питер, 2003. - 460 с. - (Учебник для вузов). - ISBN 5-94723-568-4.
10. Бесплатные образовательные программы партнера (VK): https://education.vk.company/students